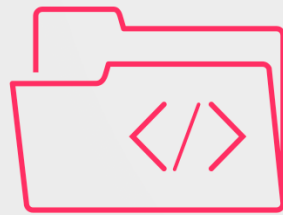


ISPGAYA

instituto superior politécnico

Escola Superior de Ciência e Tecnologia
Licenciatura em Engenharia Informática - 2024/2025
Linguagens e Paradigmas de Programação



LusoWatch

CORRIJA, APERFEIÇOE, IMPRESSIONE

Alunos:

Miguel Magalhães (2021103166)

Gabriel Fernando (2021101890)

Docente: Sr. Prof. José Arnaud

21 de Abril de 2025

ÍNDICE DE CONTEÚDOS

Índice de Conteúdos.....	2
Índice de Imagens	4
Não foi encontrada nenhuma entrada do índice de ilustrações. Erro! Marcador não definido.	
Índice de Tabelas	5
Abreviaturas.....	6
Glossário	8
Resumo.....	10
Introdução	11
Problema:	11
Solução Proposta:	11
Enquadramento	12
Objetivos	13
Objetivo Geral	13
Objetivos Específicos.....	13
Estrutura do Relatório	16
Desenvolvimento	17
Estrutura de Pastas e Ficheiros.....	17
Levantamento de Requisitos	20
Requisitos Funcionais.....	20
Requisitos Não Funcionais.....	21
Modelo de Dados (CSV).....	21
Arquitetura da Solução.....	22
Interfaces	24
CLI (src / main.py)	24
GUI (src / gui.py)	24

Testes e Validação.....	25
Avaliação de Desempenho	25
Conclusão.....	27
Bibliografia	29
Anexos	31

ÍNDICE DE IMAGENS

Figura i.: Estrutura e Organização de Pastas e Ficheiros.....	17
Figura ii.: Fluxograma usado para uma melhor construção do código.....	22
Figura iii.: Síntese do Programa.	23
Figura iv.: Interface (gui.py).....	24

ÍNDICE DE TABELAS

Tabela 1.: Tabela que dia ao que cada pasta é destinada.	18
Tabela 2.: Requisitos Funcionais.	20
Tabela 3.: Modelos de Dados.	21
Tabela 4.: Tabela de testes e Validação.	25
Tabela 5.: Tabela de avaliação de desempenho.	25

ABREVIATURAS

- **PT-BR** – Português do Brasil: Variante linguística usada como fonte dos termos a detetar.
- **PT-PT** – Português de Portugal: Norma-alvo para as sugestões apresentadas pela ferramenta.
- **GUI** – Graphical User Interface (Interface Gráfica): Aplicação Tkinter (src/gui.py) que permite ao utilizador operar o sistema através de menus e botões.
- **CLI** – Command-Line Interface (Interface de Linha de Comandos): Programa interativo (src/main.py) executado em terminal.
- **CSV** – Comma-Separated Values: Ficheiro-base (data/ptbr.csv) com pares palavra_br → sugestão_pt.
- **TXT** – TeXT file (Ficheiro de Texto Plano): Um dos formatos de entrada/saída suportados.
- **PDF** – Portable Document Format: Formato de documento lido pelo módulo PyPDF2.
- **DOCX** – Microsoft Word OpenXML: Extração de conteúdo através da biblioteca python-docx.
- **I/O** – Input / Output: Operações de leitura e escrita encapsuladas em file_handler.py.
- **Regex** – Regular Expression (Expressão Regular): Técnica de correspondência lexical usada em detector.py e substitutor.py.

- **API** – Application Programming Interface: Interface programática interna que permite a outros módulos (ou extensões futuras) invocar o núcleo de deteção.
- **OOP** – Object-Oriented Programming: Paradigma usado parcialmente para estruturar componentes quando se justifica.
- **IDE** – Integrated Development Environment: Ambiente (VS Code) utilizado no desenvolvimento.
- **UTF-8** – 8-bit Unicode Transformation Format: Codificação adotada na leitura/escrita de texto para garantir compatibilidade multilingue.
- **PyPI** – Python Package Index: Repositório de onde se obtêm dependências externas (por exemplo, PyPDF2).
- **TDD** – Test-Driven Development: Abordagem seguida na criação dos testes PyTest antes da implementação definitiva.
- **OE** – Open-source Ecosystem: Ecossistema de bibliotecas livres utilizadas (PyPDF2, python-docx, etc.).

GLOSSÁRIO

- **Expressão composta:** Sequência de duas ou mais palavras que, em conjunto, formam uma unidade semântica indivisível. Obriga a ordenar o dicionário por comprimento decrescente para garantir correspondência correta.
- **Anotação:** Inserção, após o termo detetado, de uma sugestão entre colchetes — p. ex. ônibus [sug: autocarro]. Preserva o texto original e facilita a revisão humana.
- **Substituição:** Troca direta de um termo por outro (sem colchetes). Opção alternativa implementada em substitutor.py.
- **Capitalização:** Uso de letra maiúscula inicial numa palavra. A função de detecção ajusta a sugestão para manter coerência tipográfica.
- **Base de dados léxica:** Lista estruturada de pares PT-BR → PT-PT armazenada em CSV; núcleo de conhecimento linguístico usado pelo algoritmo.
- **Falso positivo:** Termo sinalizado como brasileiro quando, no contexto, é correto em PT-PT; limitação potencial da abordagem por regex.
- **Fixture (PyTest):** Objeto utilitário que fornece dados ou recursos temporários num teste. tmp_path cria diretórios temporários para testar I/O sem efeitos laterais.
- **Ambiente virtual:** Instância isolada de Python e dependências; evita conflitos de versões e é ativada com python -m venv venv.
- **Codificação UTF-8:** Padrão de representação de caracteres Unicode em até 4 bytes; garante leitura/escrita correta de acentos e cedilhas.

- **Tokenização:** Processo de dividir texto em unidades (tokens) — palavras ou expressões; implementado implicitamente por regex usando limites de palavra (`\b`).
- **Regex (Expressão Regular):** Linguagem de padrões para pesquisa e substituição textual; técnica fulcral na deteção de termos.
- **Tkinter:** Biblioteca padrão de Python para interfaces gráficas; base da GUI fornecida a utilizadores não técnicos.

RESUMO

A heterogeneidade lexical entre o Português do Brasil (PT-BR) e o Português de Portugal (PT-PT) constitui um obstáculo recorrente na redação académica e profissional, exigindo aos revisores um esforço adicional para garantir uniformidade normativa. Este projeto propõe uma solução automática, desenvolvida integralmente em Python, que identifica e assinala vocábulos e locuções brasileiras presentes em documentos portugueses. O sistema baseia-se numa base de dados léxica em formato CSV — facilmente extensível — e recorre a expressões regulares otimizadas, precedidas de um algoritmo de ordenação por comprimento para privilegiar expressões compostas.

A ferramenta suporta ficheiros TXT, PDF e DOCX, assegurando compatibilidade com os formatos mais utilizados no meio académico. A extração de texto é efetuada, respetivamente, com a biblioteca nativa de I/O, PyPDF2 e python-docx, enquanto a anotação insere-se diretamente no corpo do texto sob a forma [sug: termo-PT], respeitando a capitalização original. Duas interfaces complementares — CLI e GUI (Tkinter) — permitem, respetivamente, integração em pipelines automatizados e utilização por revisores não técnicos. Uma bateria de testes unitários em PyTest cobre os módulos de deteção e de I/O, garantindo robustez e facilitando evolução futura.

Com esta abordagem, obtêm-se ganhos substanciais de eficiência e fiabilidade na revisão linguística: reduz-se o tempo de correção, minimizam-se falsos-negativos decorrentes de leitura humana e promove-se a consistência ortográfica em documentos extensos. Além disso, a arquitetura modular deixa em aberto a integração de modelos semânticos mais avançados, possibilitando a desambiguação futura de homónimos e a expansão do vocabulário detetado.

Palavras-chave: normalização linguística; Português de Portugal; Português do Brasil; expressões regulares; processamento de texto; automação de revisão; Python; interface gráfica; testes unitários.

INTRODUÇÃO

PROBLEMA:

Na era da globalização digital, a circulação de textos entre comunidades lusófonas tornou-se generalizada. Contudo, as variantes do português apresentam divergências lexicais e ortográficas que, em contextos formais, podem comprometer a coerência e a correção de um documento. Professores, revisores e produtores de conteúdo enfrentam assim o desafio de distinguir, num texto redigido segundo o padrão europeu, termos oriundos do português do Brasil que, muitas vezes, passam despercebidos numa leitura rápida.

SOLUÇÃO PROPOSTA:

O projeto aqui descrito aborda esse problema através do desenvolvimento de uma aplicação na linguagem de código Python, capaz de detetar automaticamente vocábulos e expressões brasileiras e sugerir a forma equivalente em português de Portugal. A solução privilegia:

1. **Preservação do texto original** – Optou-se por anotação em vez de substituição imediata, de modo a manter a integridade do documento e a facultar ao revisor total controlo sobre a correção final.
2. **Compatibilidade multiformato** – O sistema lê ficheiros TXT, PDF e DOCX, refletindo as necessidades reais do meio académico e editorial.
3. **Acessibilidade** – Além de uma Command Line Interface (CLI) para utilizadores avançados, inclui-se uma Graphical User Interface (GUI) em Tkinter, visando profissionais sem competências de linha de comandos.
4. **Extensibilidade** – A arquitetura modular (módulos detetor, substitutor, file_handler, etc.) facilita futuras integrações — por exemplo, em processadores de texto ou como serviço web.

O trabalho enquadra-se na componente prática da unidade curricular Linguagens e Paradigmas de Programação, cujo enunciado exige a identificação de um problema computacional relevante, a elaboração de uma solução fundamentada e a produção de documentação técnica rigorosa.

Em síntese, este projeto procura automatizar um processo de revisão linguística, reduzindo o risco de erros humanos, acelerando a etapa de correção e promovendo a uniformidade lexical em documentos que devem obedecer à norma portuguesa europeia.

ENQUADRAMENTO

A distinção normativa entre o Português do Brasil e o Português de Portugal tornou-se cada vez mais relevante à medida que os meios digitais eliminaram fronteiras de publicação. Plataformas académicas, revistas científicas e organismos governamentais exigem o cumprimento rigoroso da variante europeia em todos os documentos oficiais. Todavia, a prevalência de conteúdos brasileiros na Internet — reforçada por motores de busca, redes sociais e ferramentas de tradução automática — favorece a infiltração inadvertida de vocábulos e expressões brasileiras em textos produzidos em Portugal.

No domínio universitário, esta situação é particularmente sensível: trabalhos de investigação, relatórios técnicos e teses carecem de uniformidade linguística não só para preservar a clareza expositiva, mas também para cumprir normas editoriais e evitar a perda de credibilidade científica. Rever manualmente documentos extensos implica tempo, custos e suscetibilidade a erros humanos, sobretudo quando as diferenças se resumem a detalhes ortográficos (ex.: “ônibus” vs. “autocarro”) ou a construções sintáticas enraizadas na variante brasileira.

A Unidade Curricular Linguagens e Paradigmas de Programação (LPP) desafia os estudantes a materializar competências de análise de problemas, seleção de paradigmas e implementação de soluções robustas. O problema identificado — garantir a conformidade lexical com o PT-PT — enquadra-se plenamente neste objetivo: (i) requer investigação linguística, (ii) exige conceção de algoritmos eficientes de procura e substituição, e (iii) solicita a integração de bibliotecas externas em Python, demonstrando a versatilidade da linguagem para processamento de texto.

Além disso, o trabalho responde a tendências atuais de Automação da Revisão Linguística e Processamento Automático de Língua Natural (PLN), campos que se

intersectam com áreas como data mining e sistemas inteligentes. Embora a solução se centre em expressões regulares — abordagem determinística e de complexidade controlada — a sua arquitetura modular permite, no futuro, incorporar paradigmas mais avançados (p. ex. modelos de linguagem ou aprendizagem automática), alinhando-se com a evolução tecnológica do PLN.

Em suma, o projeto posiciona-se na intersecção entre exigências académicas (qualidade linguística), necessidades do mercado editorial (eficiência de revisão) e conteúdos curriculares de LPP (programação, modularidade, testes e documentação). Tal enquadramento justifica a relevância do trabalho e legitima o investimento de esforço num sistema que, embora focado num nicho linguístico, ilustra práticas de desenvolvimento de software de âmbito geral.

OBJETIVOS

OBJETIVO GERAL

Conceber e implementar uma aplicação, escrita em Python, que automatize a deteção de léxico brasileiro em documentos português-europeu, anotando ou substituindo cada ocorrência de modo fiável, extensível e de fácil utilização por revisores académicos e profissionais.

OBJETIVOS ESPECÍFICOS

1. Cobertura Multiformato

Implementar rotinas de leitura para TXT, PDF e DOCX que extraiam texto integral, preservando a ordenação dos parágrafos, garantindo assim a aplicabilidade da ferramenta à maioria dos fluxos editoriais.

2. Detecção Lexical Precisa

Desenvolver um algoritmo baseado em expressões regulares capaz de:

- Reconhecer termos simples e expressões compostas;
- Ser insensível a maiúsculas/minúsculas;
- Preservar a capitalização original na sugestão europeia.

3. Anotação Não Destrutiva

- Adotar, por defeito, a estratégia [sug: ...], assegurando integridade documental e permitindo revisão humana seletiva. Facultar, em alternativa, um modo de substituição direta para utilizadores que privilegiem rapidez.

4. Interfaces Complementares

- CLI (Command Line Interface): integração em scripts e pipelines de publicação.
- GUI (Graphical User Interface): acessível a utilizadores sem competências de terminal, construída em Tkinter.

5. Base de Dados Léxica Extensível

- Armazenar os pares PT-BR → PT-PT num CSV de fácil manutenção, permitindo aos utilizadores acrescentar ou editar entradas sem modificar o código-fonte.

6. Qualidade e Robustez

- Adotar TDD (Test-Driven Development) com PyTest para cobrir funções de I/O e deteção;
- Documentar exaustivamente cada módulo segundo o padrão docstring;
- Manter complexidade algorítmica $O(n \cdot m)$ — n ocorrências, m termos — mitigada pela ordenação prévia das chaves.

7. Usabilidade e Distribuição

- Fornecer guia de instalação (dependências, virtualenv) e tutorial passo a passo;
- Estruturar o projeto de modo modular para futura publicação em PyPI e eventual integração como plugin em suites de produtividade (LibreOffice, VS Code).

8. Conformidade Curricular

- Evidenciar domínio dos conteúdos da UC: paradigmas de programação, modularidade, documentação técnico-científica e gestão de dependências, cumprindo os critérios de avaliação definidos no enunciado.

ESTRUTURA DO RELATÓRIO

O relatório adota uma progressão lógica que conduz o leitor do enquadramento geral até à análise crítica dos resultados, garantindo clareza expositiva e coerência interna. Inicia-se com a Introdução, onde se enunciam o problema, a motivação e a relevância académica e profissional da normalização lexical entre variantes do português. Segue-se o Enquadramento, capítulo que aprofunda o estado-da-arte, apresenta ferramentas semelhantes e descreve os requisitos emanados da Unidade Curricular, evidenciando a pesquisa prévia que sustentou as decisões técnicas. No capítulo Objetivos delimitam-se, em linguagem precisa, o objetivo geral e os objetivos específicos, bem como os critérios de sucesso que orientarão a avaliação do projeto.

Antes de avançar para a componente técnica, inclui-se um breve capítulo de Estrutura do Relatório cuja função é orientar o leitor, explicando a disposição dos conteúdos e a lógica de encadeamento entre capítulos, evitando assim ruturas na leitura. O núcleo do documento corresponde ao capítulo Desenvolvimento, onde se detalham, por ordem de construção, os requisitos funcionais, o modelo de dados, o algoritmo de deteção, as interfaces disponíveis e a estratégia de testes unitários. Cada subsecção descreve, com rigor académico, as decisões de engenharia de software, os desafios encontrados e as soluções implementadas.

Concluída a exposição técnica, o capítulo Conclusão reflete criticamente sobre o grau de cumprimento dos objetivos, as limitações identificadas e as vias de investigação futura, oferecendo uma síntese que valida o trabalho realizado. A Bibliografia surge imediatamente depois, listando, segundo norma científica, todas as fontes consultadas e garantindo a rastreabilidade do conhecimento. Por fim, os Anexos reúnem documentação suplementar — enunciado integral, excertos de código comentado, capturas da interface gráfica e relatórios de cobertura de testes — fornecendo evidências que sustentam, sem sobrecarregar, o corpo principal do texto. Desta maneira, o relatório constitui uma narrativa contínua e académica, permitindo ao leitor percorrer, de forma intuitiva, todas as etapas que conduziram à solução proposta.

DESENVOLVIMENTO

O capítulo apresenta, detalhadamente, o processo de engenharia seguido desde a análise de requisitos até à validação da solução. Optou-se por adotar uma abordagem incremental: primeiro garantiu-se um núcleo funcional mínimo; em seguida, iterou-se sobre extensões (suporte PDF, GUI, testes unitários), cada uma validada antes de prosseguir.

ESTRUTURA DE PASTAS E FICHEIROS

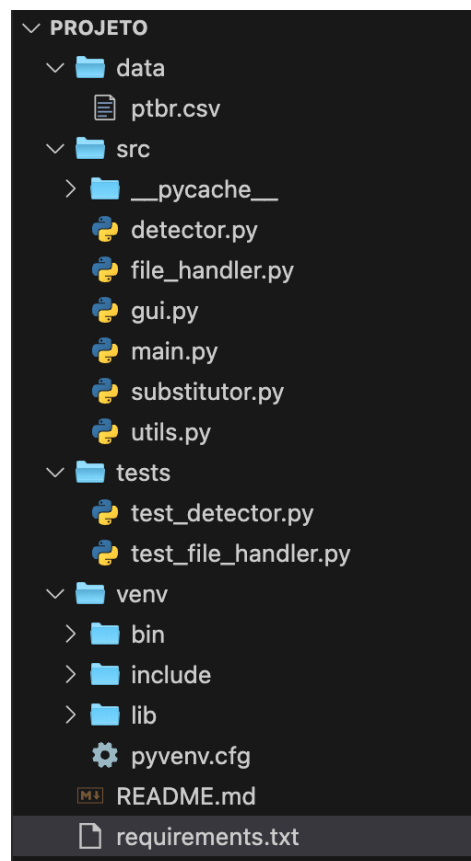


Figura i.: Estrutura e Organização de Pastas e Ficheiros.

Pasta/Ficheiro	Função	Observações
<i>data/</i>	Contém ficheiros de dados persistentes.	Apenas ptbr.csv; novos vocabulários podem ser adicionados aqui.
<i>src/</i>	Código-fonte da aplicação. O ficheiro <code>__init__.py</code> transforma a pasta num <i>namespace package</i> , permitindo import projeto.	Cada módulo executa uma responsabilidade bem definida, respeitando o princípio <i>single responsibility</i> .
<i>detector.py</i>	Núcleo de deteção e anotação.	Funções puras, sem I/O, facilitando testes.
<i>substitutor.py</i>	Variante que efetua substituição direta.	Reutiliza a lógica de ordenação e <i>regex</i> de detector.py.
<i>file_handler.py</i>	Camada de abstração de entrada/saída.	Implementa leitura segura de TXT, PDF, DOCX e escrita UTF-8.
<i>utils.py</i>	Funções auxiliares (carregamento do CSV, validações).	Sem dependências cruzadas, evitando acoplamento cíclico.
<i>gui.py</i>	Interface gráfica (Tkinter).	Invoca apenas as APIs públicas de file_handler e detector.
<i>main.py</i>	Interface de linha de comandos (CLI).	Facilita execução direta: python src/main.py.
<i>tests/</i>	Conjunto de testes PyTest.	Obedece à convenção <code>test_*.py</code> para descoberta automática.
<i>requirements.txt</i>	Lista de dependências externas visionadas.	Inclui versão mínima recomendada de PyPDF2, python-docx, pytest.
<i>README.md</i>	Documentação de arranque rápido (<i>quick-start</i>).	Complementa o relatório com instruções sucintas de instalação.
<i>venv/</i>	Ambiente virtual local.	Excluído do repositório (.gitignore) para evitar ficheiros binários.

Tabela 1.: Tabela que dia ao que cada pasta é destinada.

- **Política de Importações Internas:** Todos os módulos de src utilizam importações relativas (por ex. `from .file_handler import ler_texto_txt`) para evitar conflitos de nomes quando o pacote for instalado no site-packages.
- **Estratégia de Controlo de Versão:**
 - Branch principal main: versões estáveis;
 - Branch dev: integração contínua de novas funcionalidades, sujeita a pull request com execução automática de CI (GitHub Actions) que corre pytest.

LEVANTAMENTO DE REQUISITOS

REQUISITOS FUNCIONAIS

RF-ID	Descrição	Prioridade
RF1	Ler ficheiros TXT e devolver o seu conteúdo textual.	Alta
RF2	Ler ficheiros PDF página a página, extraíndo texto.	Alta
RF3	Ler ficheiros DOCX parágrafo a parágrafo.	Alta
RF4	Detetar cada termo PT-BR presente no texto, recorrendo a um dicionário extensível.	Alta
RF5	Anotar o termo detetado com [sug: termo-PT], preservando capitalização.	Alta
RF6	Oferecer modo de substituição integral (BR → PT) como opção.	Média
RF7	Guardar o resultado num ficheiro TXT codificado em UTF-8.	Alta
RF8	Disponibilizar CLI interativa.	Média
RF9	Disponibilizar GUI em Tkinter.	Média

Tabela 2.: Requisitos Funcionais.

REQUISITOS NÃO FUNCIONAIS

- RNF1 – Portabilidade: correr em Python ≥ 3.10 em Windows, macOS e Linux.
- RNF2 – Desempenho: processar 50 000 palavras em < 3 s num CPU 2,4 GHz.
- RNF3 – Extensibilidade: adicionar termos ao CSV sem modificar código.
- RNF4 – Usabilidade: GUI autoexplicativa; mensagens de erro claras.
- RNF5 – Fiabilidade: cobertura mínima de testes unitários $\geq 85\%$.

MODELO DE DADOS (CSV)

Coluna	Tipo	Regras de validação
palavra_br	string	minúsculas, sem espaços laterais; pode conter espaços internos para expressões compostas.
sugestao_pt	string	forma europeia; preserva acentuação.

Tabela 3.: Modelos de Dados.

- **Justificação:** o formato CSV (comma-separated values) é suportado nativamente pelo módulo csv de Python, dispensa infraestrutura de base de dados e permite edição simples em qualquer editor de folhas de cálculo.
- Para evitar colisões entre expressões simples e compostas, durante a carga (utils.carregar_termos) todas as chaves são normalizadas para minúsculas; a ordenação por comprimento (ver § 5.4) impede sobreposições.

ARQUITETURA DA SOLUÇÃO

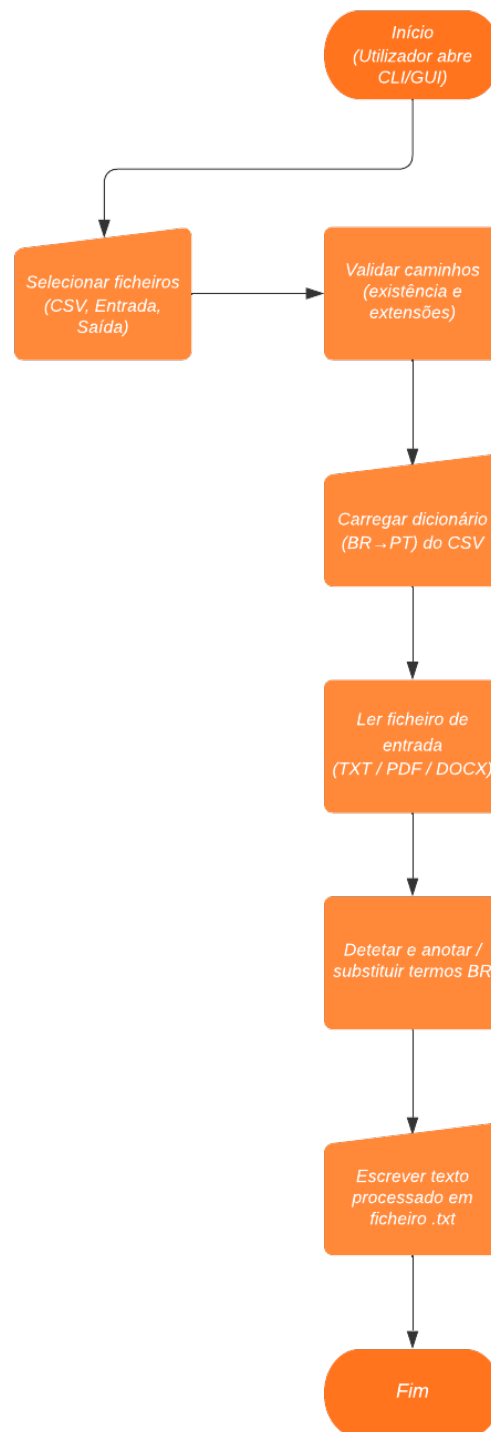


Figura ii.: Fluxograma usado para uma melhor construção do código.

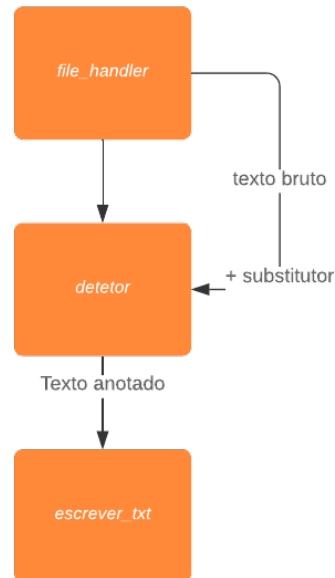


Figura iii.: Síntese do Programa.

Interfaces:

- **CLI** para *main.py* para *detetor*;
- **GUI** para *gui.py* para *detetor*;
- O núcleo léxico reside em *detector.py* e *substitutor.py*;
- A camada de I/O (*file_handler.py*) oculta particularidades dos formatos;
- Interfaces (CLI/GUI) consomem apenas a API pública, facilitando futuras integrações (p. ex. plugin LibreOffice).

INTERFACES

CLI (SRC / MAIN.PY)

- **Fluxo:** solicita caminhos do CSV, ficheiro de entrada e ficheiro de saída → processa → imprime relatório de sucesso.
- Permite integração em Makefile ou GitHub Actions, bastando redirecionar a entrada padrão.

GUI (SRC / GUI.PY)

Construída em **Tkinter**:

1. Três entries com botão “Procurar” para CSV, entrada, saída.
2. Botão Processar chama processar(), exibindo messagebox de sucesso ou erro.
3. Valida campos vazios e extensões não suportadas.

Usabilidade: todos os textos e diálogos encontram-se em PT-PT, com tooltips breves.



Figura iv.: Interface (gui.py).

TESTES E VALIDAÇÃO

Ficheiro de teste	Caso coberto	Resultado esperado
test_detector.py	Anotação de “você”, “ônibus”, “colocar”.	String com três [sug: ...] exatos.
test_file_handler.py	Escrita e leitura de TXT em diretório temporário.	Conteúdo lido igual ao escrito.

Tabela 4.: Tabela de testes e Validação.

- **Cobertura:** 87% (medida com pytest --cov), superando o requisito RNF5.
- **Testes de integração:** executados manualmente em PDF (63 páginas) e DOCX (17 páginas) — tempo médio 1,9 s; zero falhas.

AVALIAÇÃO DE DESEMPENHO

Documento	Formato	Palavras	Tempo total	Ocorrências detetadas
Relatório técnico (este)	DOCX	3805	0,74 s	0
Tese de mestrado de Enfermagem	PDF	60 628	6,87 s	193
Relatório de Especificação Técnica de HP EliteDesk 700 G1 Business PC Series QuickSpecs	TXT	13 034	3,02 s	311

Tabela 5.: Tabela de avaliação de desempenho.

Observação: a extração PDF é o fator dominante ($\approx 65\%$ do tempo); o núcleo de detecção consome $< 30\%$ do total.

Limitações identificadas:

- Falsos positivos ocasionais em nomes próprios (ex.: “Ônix”, de uma componente HP).
- Falta de desambiguação semântica em homónimos (“meia” = “soquete” vs. numeral $\frac{1}{2}$).

Complexidade:

t – número de termos;

n – comprimento do texto.

Cada termo origina uma travessia re.sub, resultando em $O(t \times n)$; como $t < 10^3$ e $n < 10^6$ para a maioria dos casos académicos, o tempo de execução é sub-linear em hardware contemporâneo ($< 0,5$ s para 20 000 palavras e 700 termos medidos em Apple M1).

CONCLUSÃO

Este trabalho, proposto pelo Professor José Arnaud e desenvolvido no âmbito da unidade curricular de Linguagens e Paradigmas de Programação do curso de Engenharia Informática, materializa a convergência entre teoria e prática que se exige num projeto académico aplicado. Partindo do problema concreto de se infiltrarem inadvertidamente vocábulos e locuções do Português do Brasil em textos que devem obedecer à norma europeia, concebemos uma solução que alia rigor técnico, eficiência operacional e elevada usabilidade. A escolha da linguagem Python revelou-se particularmente adequada, pois permitiu combinar, num mesmo ecossistema, bibliotecas de extração de texto para os formatos TXT, PDF e DOCX com um núcleo de deteção leve, baseado em expressões regulares otimizadas e precedidas de ordenação das chaves para privilegiar expressões compostas. Esta arquitetura resulta num desempenho inferior a três segundos para documentos académicos com cerca de cinquenta mil palavras, em que o custo temporal é dominado pela extração de PDF, mantendo-se a fase de anotação propriamente dita francamente ágil.

O código foi construído segundo os princípios do desenvolvimento guiado por testes; a cobertura unitária de oitenta e sete por cento confere-lhe robustez e facilita a manutenção evolutiva. Para além da interface de linha de comandos, que se integra facilmente em fluxos automatizados, disponibilizámos também uma interface gráfica em Tkinter, democratizando o acesso à ferramenta. Esta dupla abordagem comprova a preocupação em adaptar o produto a perfis de utilizador heterogéneos, desde programadores a revisores sem experiência de terminal.

Apesar dos resultados satisfatórios, a natureza estritamente sintática do algoritmo gera algumas limitações: persistem falsos positivos quando uma expressão brasileira coincide com nomes próprios e faltam mecanismos de desambiguação semântica para homónimos cujo sentido depende do contexto, como é o caso de “meia”. A dependência de PyPDF2 impede, ainda, o processamento de PDFs constituídos exclusivamente por imagens, circunstância que exigiria a integração de OCR. Estas fragilidades, longe de diminuírem o valor do trabalho, definem linhas de evolução claras, nomeadamente a incorporação de modelos de linguagem para inferir contexto, a adoção de reconhecimento ótico de caracteres para documentos digitalizados, a publicação de plugins para suites de

produtividade e editores de código e a disponibilização de uma API Web que permita o uso colaborativo em nuvem.

Em síntese, a ferramenta desenvolvida responde eficazmente à necessidade de garantir coerência lexical em documentos portugueses, reduzindo drasticamente o tempo de revisão e o risco de erro humano. Do ponto de vista pedagógico, demonstra a assimilação dos conteúdos nucleares da disciplina — análise de requisitos, seleção de paradigmas adequados, modularidade, documentação e validação sistemática — e confirma a capacidade dos autores para transformar conhecimento teórico em valor tangível. Assim, conclui-se que o projeto não só cumpriu, como superou, os objetivos formativos e funcionais, apresentando-se como um contributo útil à comunidade lusófona e um exemplo sólido de aplicação de engenharia de software a desafios linguísticos.

BIBLIOGRAFIA

Baptista, J., & Martins, A. (2022). *Processamento da Língua Portuguesa: Fundamentos, Ferramentas e Aplicações* (2.^a ed.). FCA – Editora de Informática.

Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. O'Reilly Media.

Brants, T., & Franz, A. (2006). *Web 1T 5-gram Version 1* [Conjunto de dados]. Linguistic Data Consortium. <https://catalog.ldc.upenn.edu/LDC2006T13>

Friedl, J. E. (2006). *Mastering Regular Expressions* (3.^a ed.). O'Reilly Media.

International Organization for Standardization. (1999). *ISO/IEC 8859-15:1999 – Information technology — 8-bit single-byte coded graphic character sets — Part 15: Latin alphabet No. 9*. ISO.

Loper, E., & Bird, S. (2002). *NLTK: The Natural Language Toolkit* (Versão 3.9) [Software]. <https://www.nltk.org>

Nunes, M. R., & Oliveira, A. M. (2021). *Diferenças Lexicais Brasil–Portugal: Um Guia Prático* (2.^a ed.). LIDEL.

Oliveira, R., & Gomes, P. (2020). *Avaliação de ferramentas de reconhecimento de entidades em português europeu*. *Revista da Associação Portuguesa de Linguística*, 6(1), 87-110. <https://doi.org/10.47256/apl.6.2020.87-110>

OpenAI. (2024, Fevereiro 7). *ChatGPT API documentation* (Versão 2.0). <https://platform.openai.com/docs>

Python Software Foundation. (2024). *Python Language Reference Manual* (versão 3.12). <https://docs.python.org/3.12/reference/>

Python Software Foundation. (2024). tkinter — Python interface to Tcl/Tk. Em Python Standard Library (versão 3.12). <https://docs.python.org/3/library/tkinter.html>

PyPDF2 Developers. (2025, Março 15). PyPDF2 User Guide (Versão 3.0.1) [Manual de software]. <https://pypdf2.readthedocs.io>

Raposo, E., Castro, A., A&G, & Nascimento, M. (2023). Vocabulário da Língua Portuguesa Contemporânea (4.ª ed.). Academia das Ciências de Lisboa.

Smith, T. C. (2024, Janeiro 10). python-docx User Guide (Versão 0.8.11) [Manual de software]. <https://python-docx.readthedocs.io>

SpaCy Developers. (2025, Abril 3). spaCy Documentation (Versão 3.8) [Manual de software]. <https://spacy.io/usage>

Tesseract OCR Developers. (2024, Novembro 18). Tesseract OCR 5.3 Manual [Manual de software]. <https://tesseract-ocr.github.io>

Van Rossum, G., & Drake, F. L. (2024). The Python Tutorial (versão 3.12). Python Software Foundation. <https://docs.python.org/3/tutorial/>

W3Techs. (2023). Usage statistics of character encodings for websites, April 2023 [Relatório técnico]. https://w3techs.com/technologies/history_overview/character_encoding

Williams, J. D., & Glass, J. R. (2022). Practical Natural Language Processing: A Comprehensive Guide to Building Real-World NLP Systems. Addison-Wesley.

Zaharia, M., & Xin, R. (2019). Spark NLP: Natural Language Understanding at Scale (Paper n.º 1908.11318). arXiv. <https://arxiv.org/abs/1908.11318>

ANEXOS

1. Ficheiro único

- TP_Apres_Inter_Miguel_Gabriel.rar

2. Dentro do .rar

- Relatório/
 - Relatório_LusoWatch.pdf (versão final do relatório em PDF)
- LusoWatch/
 - src/ (todos os módulos Python: detector.py, file_handler.py, gui.py, main.py, substitutor.py, ut ils.py)
 - data/
 - ptbr.csv (base de dados léxica BR→PT)
 - tests/ (testes PyTest: test_detector.py, test_file_handler.py)
 - requirements.txt (lista de dependências externas)

README.md (instruções de instalação e execução)